UNIVERSIDAD NACIONAL

SEDE REGIONAL CHOROTEGA CAMPUS NICOYA

CARRERA: INGENIERÍA EN SISTEMAS DE LA INFORMACIÓN

Paradigmas de Programación

Laboratorio 5

Persona académica: Ing. Alex Villegas Carranza. M.Sc.

Correo electrónico: alex.villegas.carranza@una.cr

Valor del laboratorio: 2 % de la calificación total del curso.

Costa Rica, Guanacaste, 2026

Lab 5 Objetivos

- Ejecutar y comprender el funcionamiento del proyecto UNA Chat.
- Identificar y solucionar vulnerabilidades de inyección de scripts.
- Implementar la funcionalidad para agregar y visualizar URLs de videos e imágenes en el chat.
- Mejorar la validación de mensajes para prevenir inyecciones de scripts creando primero pruebas unitarias de seguridad.
- Mejorar la validación de mensajes para permitir URLs de imágenes y videos de manera segura.
- Realizar pruebas unitarias para validar URLs de imágenes y videos.
- Realizar pruebas de integración entre el backend Node.js y la librería **una-lib**.
- Realizar pruebas end-to-end con **Playwright** para comprobar el flujo completo entre el frontend y el backend.

UNA Chat cuenta con un frontend y un backend. Las pruebas unitarias revisan la lógica de validación; las de integración, la colaboración del backend con **una-lib**; y las end-to-end, el recorrido completo desde el navegador.

Instrucciones

1 Ejecutar el Proyecto UNA Chat

- Abra la terminal y navegue hasta el directorio raíz del proyecto UNA Chat.
- Ejecute el comando:

```
npm start
```

- Verifique que la aplicación se esté ejecutando correctamente en su navegador.

2 Ejecutar un Script Injection

- En el campo de texto del chat, ingrese un script malicioso, por ejemplo:

```
<script>alert('Inyección de script');</script>
```

Observe si el script se ejecuta. Si es así, la aplicación es vulnerable a inyecciones de scripts.

3 Tomar captura de esto como evidencia 1

4 Agregar Funcionalidad para URLs en el Chat

- Modifique el código del chat para permitir que los usuarios ingresen URLs en el campo de texto.
- Implemente la lógica para detectar si la URL es de una imagen o un video:
  - Si es una imagen, muestre la imagen directamente en el chat.
  - Si es un video, inserte un reproductor de video en el chat.

5 Mejorar la Validación en UNA Lib

- En el método **validarMensaje** dentro de la librería **una-lib**:
  - Agregue validaciones para prevenir inyecciones de scripts y eliminar cualquier código malicioso.
  - Permita únicamente URLs que apunten a imágenes o videos válidos.
  - Utilice expresiones regulares o bibliotecas de validación para asegurar la integridad de los datos.

6 Ejecutar Pruebas Existentes

- En la terminal, ejecute las pruebas existentes con el comando:

```
npm test
```

7 Crear Nuevas Pruebas Unitarias

- Escriba pruebas unitarias adicionales para cubrir los nuevos casos:
  - Validación de URLs de Imágenes:
    - Asegúrese de que las URLs que apuntan a imágenes se validen y muestren correctamente.
  - Validación de URLs de Videos:
    - Verifique que las URLs de videos sean aceptadas y que el reproductor se integre adecuadamente.
  - Prevención de Inyección de Scripts:
    - Confirme que los intentos de inyección de scripts sean detectados y bloqueados por la validación.

8 Tomar Screenshot de las pruebas siendo ejecutadas y adjuntar

9 Crear Pruebas de Integración

Compruebe que el backend de Node.js y **una-lib** trabajan juntos. Se recomienda **Jest** para ejecutar las pruebas y **Supertest** para las rutas HTTP.

- En la carpeta del backend, instale estas dependencias de desarrollo. Si Jest ya está instalado, ejecute el comando únicamente con Supertest.

  ```
  npm install --save-dev jest supertest
  ```

- Ubique la ruta y el método HTTP que recibe los mensajes. Cree `tests/integration/chat.test.js` y envíe solicitudes con Supertest a esa entrada real.
- Compruebe que el backend llama realmente a **validarMensaje** de **una-lib**. Incluya los siguientes casos y revise el código de estado y el contenido de la respuesta:
  - URL de imagen válida: el backend acepta el mensaje.
  - URL de video válida: el backend acepta el mensaje.
  - URL no permitida o script: la validación rechaza la entrada o devuelve contenido saneado, según la lógica implementada.
- Ejecute la suite desde el backend y corrija los fallos. Al terminar, cierre los servidores o conexiones que abrió la prueba.

  ```
  npx jest tests/integration --runInBand --env=node
  ```

- Guarde una captura de las pruebas aprobadas como **evidencia 3**.

Si el envío usa Socket.IO: instale `socket.io-client` como dependencia de desarrollo y use Jest para probar el evento real de envío y su respuesta. Supertest se utiliza para HTTP.

10 Crear Pruebas End-to-End con Playwright

Automatice el recorrido completo del usuario con el frontend y el backend reales de UNA Chat.

- Inicie ambos componentes con los comandos de sus archivos `package.json`. Compruebe en el navegador que el chat se comunica con el backend.
- En la raíz del proyecto, configure Playwright:

```
npm init playwright@latest
```

Seleccione JavaScript, `tests/e2e` como carpeta de pruebas y la instalación de navegadores. En `playwright.config.js`, ajuste **testDir** a esa carpeta y **use.baseURL** a la URL real del frontend.

- Cree `tests/e2e/chat.spec.js`. Utilice **page.goto**, localizadores como **getByRole**, **fill**, **click** y **expect** para automatizar estos casos:
  - Enviar una URL de imagen y verificar que la imagen se muestra y termina de cargar en el chat.
  - Enviar una URL de video y verificar que el reproductor aparece y carga el contenido. Use medios de prueba conocidos y estables.
  - Enviar el script del paso 2. Registre **page.on("dialog")** antes del envío para detectar el alert y hacer fallar la prueba. Compruebe además el rechazo o saneamiento del mensaje.
- Mantenga ambos componentes activos y ejecute en Chromium:

```
npx playwright test --project=chromium --headed
```

- Corrija los fallos y guarde una captura del resultado aprobado como **evidencia 4**. Consulte el reporte con:

```
npx playwright show-report
```

11. Ejecutar un escaneo de vulnerabilidades en dependencias con la extensión de Snyk.

El comando snyk test debería realizar el escaneo y encontrar una lista de vulnerabilidades, repara todas las criticas y altas usando un agente de IA, y luego vuelve a correr todas las pruebas.




```
PS C:\Users\alexv\OneDrive\Documentos\Datos\Work Stuff\UNA Clases\2026\2 - Seguridad Informática\Lab-5\despues> snyk test

Testing C:\Users\alexv\OneDrive\Documentos\Datos\Work Stuff\UNA Clases\2026\2 - Seguridad Informática\Lab-5\despues...

Organization:      alexvillegasc
Package manager:   npm
Target file:       package-lock.json
Project name:      package.json
Open source:       no
Project path:      C:\Users\alexv\OneDrive\Documentos\Datos\Work Stuff\UNA Clases\2026\2 - Seguridad Informática\Lab-5\despues
Licenses:          enabled

✓ Tested 82 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

Bonus (Esto es solo información adicional, no es parte del Lab):

¿Como saber si las pruebas de verdad sirven? Una solución propuesta es:

https://github.com/Sweet-Papa-Technologies/Agentic-SDLC

FoFo no es un botón de “correr tests”. Es el árbitro que revisa si las pruebas de verdad sirven, y si el código las pasa. Si lo utilizas con un Spec Framework como openspec, puede incluirse como parte del flujo de trabajo.

Step	OpenSpec’s responsibility	FoFo / development responsibility
1. Explore and propose	Define the problem, requirements, scenarios, design, and tasks.	Identify acceptance criteria that can actually be tested.
2. Approve the change	Human reviews scope and expected behavior.	Map approved requirements into FoFo’s input format.
3. Prepare tests	Keep the approved specifications available as the contract.	Judge independently derives test intents, write s failing tests, and locks them.
4. Implement	Track progress against tasks.md.	Author implements against those tests; run the test-and-fix loop.
5. Verify and review	Check implementation against requirements, scenarios, and design.	Run FoFo gates, normal CI, and PR review. Route code fixes to the Author and test fixes to the Judge.
6. Complete and archive	Synchronize the completed change into the maintained specifications and archive it.	Preserve test evidence and review results with the PR.
7. Release and operate	Start new changes when requirements evolve.	Continue normal deployment, monitoring, and incident handling.

11 Entrega del Laboratorio 5

Valor: 3 % de la calificación total del curso.
Fecha de entrega: viernes 23 de octubre de 2026, 23:59:59.

Adjunte en **ProfeLink** un PDF con las capturas de cada tipo de prueba completada JPG y al inicio el enlace a su repositorio de GitHub:

- **Evidencia 1:** resultado de la prueba de inyección de scripts del paso 2.
- **Evidencia 2:** ejecución de las pruebas unitarias del paso 8.
- **Evidencia 3:** ejecución de las pruebas de integración.
- **Evidencia 4:** ejecución de las pruebas end-to-end con Playwright.
- **repositorio.txt:** enlace al repositorio con el código del proyecto, las tres suites de pruebas y la configuración necesaria para ejecutarlas.

No suba documentos de Word ni PDF. Se evaluarán las capturas JPG y el código disponible en el repositorio enlazado en el archivo .txt.

Antes de entregar

Verifique que las tres suites ejecuten las pruebas que les corresponden. Si `npm test` usa Jest, agregue `tests/e2e` a **testPathIgnorePatterns** para que los archivos de Playwright se ejecuten con su propio comando.

Documentación de apoyo

Jest: instalación y primeras pruebas
Supertest: pruebas de rutas HTTP
Socket.IO: pruebas de eventos
Playwright: instalación
Playwright: escritura de pruebas
Playwright: ejecución y reportes